

E-UNI

Espace Numérique Universitaire

DOCUMENTATION TECHNIQUE

Guide destiné aux développeurs

Stack technique : React · Node.js / Express · MySQL

Infinity Code — Port-au-Prince, Haïti

Version 1.0 — Août 2026

Table des matières

1. Présentation du projet
2. Architecture technique
3. Structure du projet
4. Modèle de données (MySQL)
5. API REST
6. Authentification et sécurité
7. Intégration MonCash
8. Installation et déploiement
9. Bonnes pratiques de développement
10. Annexes

1. Présentation du projet

E-UNI (Espace Numérique Universitaire) est une plateforme numérique destinée aux institutions universitaires haïtiennes. Elle centralise la gestion des cours et programmes académiques, le suivi des notes et évaluations, le paiement des frais académiques via MonCash, la communication interne (messenger et notifications), ainsi qu'une bibliothèque de ressources numériques accessible aux étudiants et enseignants.

Ce document s'adresse aux développeurs chargés de la conception, de la maintenance et de l'évolution de la plateforme. Il décrit l'architecture technique, le modèle de données, les API, ainsi que les procédures d'installation et de déploiement.

1.1 Objectifs de la plateforme

- Digitaliser la gestion académique (cours, programmes, notes, évaluations).
- Faciliter le paiement des frais universitaires via MonCash, adapté au contexte haïtien.
- Améliorer la communication entre administration, enseignants et étudiants.
- Offrir un accès centralisé aux ressources pédagogiques numériques.
- Fonctionner de manière fiable malgré les contraintes d'infrastructure locales (connectivité limitée).

2. Architecture technique

E-UNI repose sur une architecture trois-tiers classique, avec séparation claire entre le front-end, le back-end et la base de données.

2.1 Vue d'ensemble

```

Client (navigateur / mobile) | HTTPS / REST (JSON) | Front-end
React (SPA) | Appels API (Axios / Fetch) | Back-end Node.js /
Express (API REST) | Requêtes SQL (ORM / driver mysql2) | Base de
données MySQL | Service externe : API MonCash (paiements)
  
```

2.2 Choix technologiques

Couche	Technologie	Rôle
Front-end	React.js (+ React Router, Axios)	Interface utilisateur, SPA
Back-end	Node.js + Express.js	API REST, logique métier
Base de données	MySQL	Stockage relationnel des données
Authentification	JWT + bcryptjs	Sécurisation des sessions et mots de passe
Paiement	API MonCash (Digicel)	Paiement des frais académiques
Déploiement	Vercel (front) / VPS ou service Node (back)	Hébergement

3. Structure du projet

3.1 Front-end (React)

```
euni-frontend/ ├── public/ ├── src/ |   ├── assets/ |   ├── components/           #
Composants réutilisables (Navbar, Cards, Modals) |   ├── pages/           # Pages :
Dashboard, Cours, Notes, Paiements, Messages, Bibliotheque |   ├── context/
# AuthContext, NotificationContext |   ├── services/           # api.js (Axios),
authService, courseService, paymentService... |   ├── hooks/ |   ├── utils/ |   ├──
App.jsx |   ├── index.jsx |   .env |   package.json
```

3.2 Back-end (Node.js / Express)

```
euni-backend/ ├── src/ |   ├── config/           # db.js, moncash.js, env |   ├──
controllers/           # authController, courseController, gradeController, |   |
# paymentController, messageController, resourceController |   ├── routes/
# authRoutes, courseRoutes, gradeRoutes, paymentRoutes, ... |   ├── models/
# Modèles / requêtes SQL par entité |   ├── middlewares/           # auth.js (JWT),
errorHandler.js, rateLimiter.js |   ├── services/           # moncashService.js,
notificationService.js |   ├── utils/ |   ├── app.js |   ├── server.js |   .env |
package.json
```

4. Modèle de données (MySQL)

Le schéma relationnel ci-dessous présente les tables principales de la base de données E-UNI et leurs relations essentielles.

4.1 Tables principales

Table	Description
utilisateurs	Comptes (étudiant, enseignant, administrateur) : identité, rôle, identifiants
programmes	Programmes académiques / filières offerts par l'institution
cours	Cours rattachés à un programme, avec enseignant responsable
inscriptions	Table de liaison étudiant ↔ cours (inscription à un cours)
evaluations	Évaluations planifiées pour un cours (examen, devoir, contrôle)
notes	Notes obtenues par un étudiant à une évaluation
frais_academiques	Frais dus par étudiant (montant, échéance, statut)
paiements	Transactions de paiement (référence MonCash, statut, montant)
messages	Messagerie interne entre utilisateurs
notifications	Notifications système envoyées aux utilisateurs
ressources	Ressources numériques (documents, liens) liées à un cours ou à la bibliothèque

4.2 Détail des colonnes — tables clés

utilisateurs

Colonne	Type	Description
id	INT, PK, AUTO_INCREMENT	Identifiant unique

Colonne	Type	Description
nom / prenom	VARCHAR(100)	Identité de l'utilisateur
email	VARCHAR(150), UNIQUE	Identifiant de connexion
mot_de_passe	VARCHAR(255)	Hash bcrypt du mot de passe
role	ENUM('etudiant','enseignant','admin')	Rôle applicatif
created_at	DATETIME	Date de création du compte

COURS

Colonne	Type	Description
id	INT, PK, AUTO_INCREMENT	Identifiant unique
nom_cours	VARCHAR(150)	Intitulé du cours
programme_id	INT, FK → programmes.id	Programme rattaché
enseignant_id	INT, FK → utilisateurs.id	Enseignant responsable
credits	INT	Nombre de crédits académiques
semestre	VARCHAR(20)	Semestre concerné

paiements

Colonne	Type	Description
id	INT, PK, AUTO_INCREMENT	Identifiant unique
etudiant_id	INT, FK → utilisateurs.id	Étudiant concerné
frais_id	INT, FK → frais_academiques.id	Frais réglé
reference_moncash	VARCHAR(100)	Référence transaction MonCash
montant	DECIMAL(10,2)	Montant payé (HTG)
statut	ENUM('en_attente','reussi','echoue')	Statut de la transaction
date_paiement	DATETIME	Horodatage du paiement

5. API REST

L'API suit les conventions REST. Toutes les routes protégées requièrent un jeton JWT transmis dans l'en-tête Authorization (Bearer token).

5.1 Authentification

Méthode	Endpoint	Description
POST	/api/auth/register	Création d'un compte utilisateur
POST	/api/auth/login	Connexion, retourne un token JWT
POST	/api/auth/refresh	Rafraîchissement du token

Méthode	Endpoint	Description
POST	/api/auth/logout	Déconnexion

5.2 Cours et programmes

Méthode	Endpoint	Description
GET	/api/cours	Liste des cours
GET	/api/cours/:id	Détail d'un cours
POST	/api/cours	Créer un cours (admin/enseignant)
PUT	/api/cours/:id	Modifier un cours
DELETE	/api/cours/:id	Supprimer un cours
GET	/api/programmes	Liste des programmes académiques

5.3 Notes et évaluations

Méthode	Endpoint	Description
GET	/api/evaluations/cours/:coursId	Évaluations d'un cours
POST	/api/evaluations	Créer une évaluation
POST	/api/notes	Saisir/mettre à jour une note
GET	/api/notes/etudiant/:id	Relevé de notes d'un étudiant

5.4 Paiements (MonCash)

Méthode	Endpoint	Description
GET	/api/frais/etudiant/:id	Frais dus par un étudiant
POST	/api/paiements/initier	Initier un paiement MonCash
GET	/api/paiements/verifier/:reference	Vérifier le statut d'une transaction
POST	/api/paiements/webhook	Callback de confirmation MonCash

5.5 Communication et ressources

Méthode	Endpoint	Description
GET	/api/messages/:userId	Messages d'un utilisateur
POST	/api/messages	Envoyer un message
GET	/api/notifications/:userId	Notifications d'un utilisateur
GET	/api/ressources	Liste des ressources numériques
POST	/api/ressources	Ajouter une ressource

6. Authentification et sécurité

- Mots de passe hachés avec bcryptjs (jamais stockés en clair).

- Authentification par JWT avec expiration courte + mécanisme de rafraîchissement.
- Middleware de contrôle de rôle (étudiant / enseignant / admin) sur chaque route sensible.
- Limitation du débit (rate limiting) sur les routes d'authentification pour prévenir les attaques par force brute.
- Validation systématique des entrées côté serveur (express-validator ou équivalent).
- Connexions à la base de données via requêtes préparées (protection contre les injections SQL).
- HTTPS obligatoire en production ; variables sensibles (clés MonCash, secrets JWT) stockées dans des variables d'environnement (.env), jamais committées.

7. Intégration MonCash

Le module de paiement s'appuie sur l'API MonCash de Digicel pour permettre aux étudiants de régler leurs frais académiques directement depuis la plateforme.

7.1 Flux de paiement

1. L'étudiant sélectionne un frais à régler et déclenche le paiement.
2. Le back-end appelle l'API MonCash pour générer une transaction et redirige l'étudiant vers la page de paiement MonCash.
3. L'étudiant confirme le paiement sur son compte MonCash.
4. MonCash notifie le back-end (webhook) ou le back-end interroge le statut de la transaction.
5. Le statut du paiement est mis à jour dans la table paiements et le frais correspondant est marqué comme réglé.

7.2 Bonnes pratiques

- Toujours vérifier le statut d'une transaction côté serveur avant de valider un paiement (ne jamais se fier uniquement au retour côté client).
- Journaliser chaque tentative de paiement (succès, échec, en attente) pour faciliter le support.
- Prévoir un mode de secours (paiement manuel avec justificatif) en cas d'indisponibilité de l'API MonCash.

8. Installation et déploiement

8.1 Prérequis

- Node.js ≥ 18.x et npm
- MySQL ≥ 8.0
- Compte marchand MonCash (clés API sandbox et production)

8.2 Installation locale

```
# Back-end cd euni-backend npm install cp .env.example .env # renseigner DB_HOST,
DB_USER, DB_PASSWORD, JWT_SECRET, MONCASH_KEY... npm run migrate # création
des tables npm run dev # démarrage en développement # Front-end cd
euni-frontend npm install cp .env.example .env # renseigner REACT_APP_API_URL npm
start
```

8.3 Déploiement

- Front-end : déploiement sur Vercel (build automatique depuis le dépôt GitHub).
- Back-end : hébergement sur un VPS ou un service Node (ex. Render, Railway) avec variables d'environnement configurées côté production.
- Base de données : instance MySQL managée ou hébergée localement selon les contraintes d'infrastructure de l'institution.
- Mettre en place des sauvegardes régulières (dump MySQL automatisé).

9. Bonnes pratiques de développement

- Respecter la structure de dossiers définie (séparation controllers / routes / models / services).
- Utiliser des noms de branches Git explicites (feature/, fix/, hotfix/) et des messages de commit clairs.
- Documenter chaque nouvelle route API (méthode, paramètres, réponse) dans ce document.
- Écrire des tests pour la logique métier critique (paiements, calcul de notes).
- Gérer les erreurs de façon centralisée via un middleware errorHandler.

10. Annexes

10.1 Variables d'environnement (back-end)

```
PORT=5000 DB_HOST=localhost DB_USER=euni_user DB_PASSWORD=***** DB_NAME=euni_db
JWT_SECRET=***** MONCASH_CLIENT_ID=***** MONCASH_CLIENT_SECRET=*****
MONCASH_MODE=sandbox
```

10.2 Glossaire

Terme	Définition
JWT	JSON Web Token, format de jeton utilisé pour l'authentification
API REST	Interface de programmation respectant les principes REST
ORM	Object-Relational Mapping, couche d'abstraction entre code et base de données
Webhook	Notification HTTP automatique envoyée par un service externe (ici, MonCash)